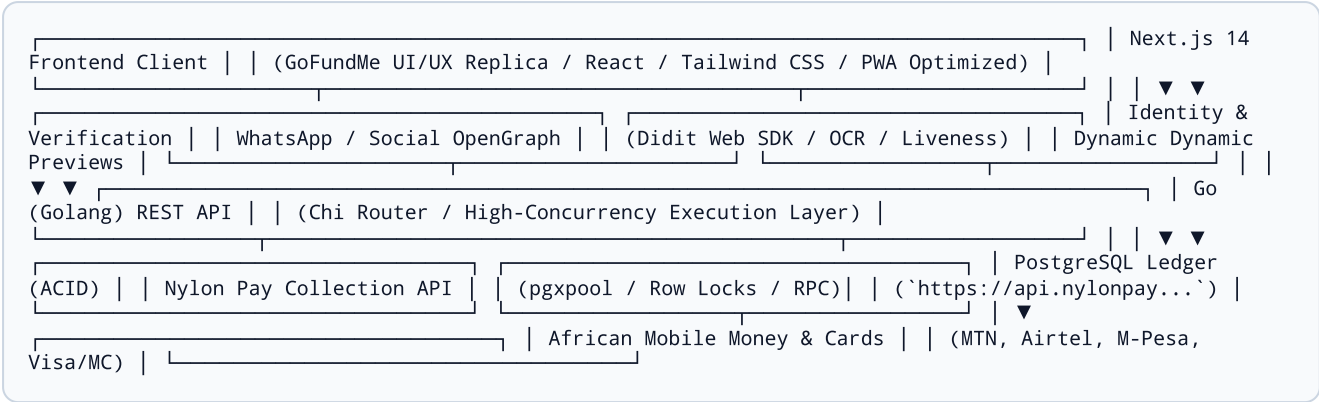


1. SYSTEM VISION & HIGH-LEVEL ARCHITECTURE

Fenna ("All of us" / "Together") is a high-concurrency, mobile-first crowdfunding platform designed specifically for Pan-African university campuses and local communities. It mirrors the high-trust visual experience of GoFundMe while natively integrating African Mobile Money rails (MTN MoMo, Airtel Money, M-Pesa) alongside automated KYC and liveness verification.



2. TECHNOLOGY STACK SPECIFICATIONS

| Layer           | Technology Choice              | Rationale & Capability                                                                                   |
|-----------------|--------------------------------|----------------------------------------------------------------------------------------------------------|
| Frontend        | Next.js 14 (App Router, TS)    | SSR support for OpenGraph previews on social platforms like WhatsApp; optimized for mobile networks.     |
| Styling & UI    | Tailwind CSS + Lucide Icons    | Direct GoFundMe visual replica (emerald green CTAs, clean typography, progress bar UI).                  |
| Backend Engine  | Go (Golang 1.22+) + Chi Router | Lightweight memory footprint, ultra-fast routing, and safe concurrency handling under viral load spikes. |
| Database        | PostgreSQL 16 (pgxpool)        | ACID compliance, explicit row-level locking, and atomic balance increment RPC stored procedures.         |
| Payment Gateway | Nylon Pay API                  | Native collection & payout rails for African Mobile Money corridors (UGX, KES, GHS, NGN, TZS).           |
| Identity / KYC  | Didit (verify.didit.me)        | Passive liveness detection, 1:1 face matching, and OCR document processing via secure webhooks.          |

### 3. DATABASE SCHEMA & LEDGER MIGRATION (`SCHEMA.SQL`)

The system uses PostgreSQL for data consistency. Atomic RPC stored procedures ensure balance integrity during high-frequency concurrent donations.

```
-- Enable Required Extensions & Domain Enums CREATE EXTENSION IF NOT EXISTS "uuid-oss"; CREATE TYPE campaign_category AS ENUM ('education', 'medical', 'emergency', 'community', 'innovation'); CREATE TYPE campaign_status AS ENUM ('draft', 'pending_review', 'active', 'paused', 'completed', 'rejected'); CREATE TYPE donation_status AS ENUM ('pending', 'successful', 'failed', 'refunded'); -- 1. Users & Verified Student Beneficiaries CREATE TABLE users ( id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), full_name VARCHAR(255) NOT NULL, email VARCHAR(255) UNIQUE NOT NULL, phone_number VARCHAR(20) UNIQUE NOT NULL, institution_name VARCHAR(255), student_reg_number VARCHAR(100), is_verified BOOLEAN DEFAULT FALSE, didit_session_id VARCHAR(255), created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW() ); -- 2. Campaigns Engine CREATE TABLE campaigns ( id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), creator_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, title VARCHAR(255) NOT NULL, slug VARCHAR(255) UNIQUE NOT NULL, category campaign_category NOT NULL, story TEXT NOT NULL, cover_image_url TEXT NOT NULL, target_amount NUMERIC(15, 2) NOT NULL CHECK (target_amount > 0), current_amount NUMERIC(15, 2) DEFAULT 0.00 CHECK (current_amount >= 0), currency VARCHAR(3) DEFAULT 'UGX', status campaign_status DEFAULT 'pending_review', created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW() ); -- 3. Immutable Donation Ledger CREATE TABLE donations ( id UUID PRIMARY KEY DEFAULT uuid_generate_v4(), campaign_id UUID NOT NULL REFERENCES campaigns(id) ON DELETE CASCADE, donor_name VARCHAR(150) DEFAULT 'Anonymous', donor_email VARCHAR(255) NOT NULL, donor_phone VARCHAR(20) NOT NULL, amount NUMERIC(15, 2) NOT NULL CHECK (amount > 0), transaction_ref VARCHAR(100) UNIQUE NOT NULL, transaction_id VARCHAR(100), status donation_status DEFAULT 'pending', is_anonymous BOOLEAN DEFAULT FALSE, created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW() ); -- Fast Lookup Indices & Stored Procedure for Safe Race-Free Balance Updates CREATE INDEX idx_donations_tx_ref ON donations(transaction_ref); CREATE OR REPLACE FUNCTION increment_campaign_balance(c_id UUID, inc_amount NUMERIC) RETURNS void AS $$ BEGIN UPDATE campaigns SET current_amount = current_amount + inc_amount, updated_at = NOW() WHERE id = c_id; END; $$ LANGUAGE plpgsql SECURITY DEFINER;
```

## 4. BACKEND API ENGINE (GOLANG HANDLERS)

Implementation of the high-performance donation workflow and HMAC-verified webhook handler for Nylon Pay Mobile Money processing.

```
// internal/handlers/donation.go - Mobile Money Payment Trigger
package handlers import ( "bytes"; "context"; "encoding/json"; "fmt"; "net/http"; "os"; "time" "fenna-backend/internal/db"; "fenna-backend/internal/models"; "github.com/google/uuid" )
func InitiateDonation(w http.ResponseWriter, r *http.Request) {
    var req models.InitiateDonationReq
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid payload", http.StatusBadRequest);
        return
    }
    ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
    defer cancel()
    txRef := fmt.Sprintf("FEN-%d-%s", time.Now().Unix(), uuid.New().String()[:6])
    _, err := db.Pool.Exec(ctx, `INSERT INTO donations (campaign_id, donor_name, donor_email, donor_phone, amount, transaction_ref, is_anonymous, status) VALUES ($1, $2, $3, $4, $5, $6, $7, 'pending')`, req.CampaignID, req.DonorName, req.DonorEmail, req.DonorPhone, req.Amount, txRef, req.IsAnonymous, )
    if err != nil {
        http.Error(w, "DB Error", 500);
        return
    }
    // Dispatch Payment Prompt to Nylon Pay Collection API
    nylonBody, _ := json.Marshal(map[string]interface{}{
        "amount": req.Amount,
        "phoneNumber": req.DonorPhone,
        "reference": txRef,
        "callbackURL": os.Getenv("APP_BASE_URL")+"/api/webhooks/nylonpay",
    })
    nylonReq, _ := http.NewRequestWithContext(ctx, "POST", "https://api.nylonpay.nilesquad.com/v1/payments/collect", bytes.NewBuffer(nylonBody))
    nylonReq.Header.Set("Authorization", "Bearer "+os.Getenv("NYLON_PAY_API_KEY"))
    client := &http.Client{Timeout: 10 * time.Second}
    resp, err := client.Do(nylonReq)
    if err != nil || resp.StatusCode >= 400 {
        http.Error(w, "Gateway Error", 502);
        return
    }
    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(map[string]interface{}{"success": true, "transactionRef": txRef})
}
```

```
// internal/handlers/nylon_webhook.go - Atomic Webhook Settlement
func HandleNylonWebhook(w http.ResponseWriter, r *http.Request) {
    rawBody, _ := io.ReadAll(r.Body)
    sig := r.Header.Get("x-nylon-signature")
    mac := hmac.New(sha256.New, []byte(os.Getenv("NYLON_PAY_WEBHOOK_SECRET")))
    mac.Write(rawBody)
    if !hmac.Equal([]byte(sig), []byte(hex.EncodeToString(mac.Sum(nil)))) {
        http.Error(w, "Invalid HMAC Signature", 401);
        return
    }
    var p models.NylonWebhookPayload
    _ = json.Unmarshal(rawBody, &p)
    if p.Status == "SUCCESSFUL" {
        tx, _ := db.Pool.Begin(r.Context())
        defer tx.Rollback(r.Context())
        var campaignID string; var amount float64
        err := tx.QueryRow(r.Context(), `UPDATE donations SET status = 'successful', nylon_transaction_id = $1 WHERE transaction_ref = $2 RETURNING campaign_id, amount`, p.TransactionID, p.Reference).Scan(&campaignID, &amount)
        if err == nil {
            _, _ = tx.Exec(r.Context(), "SELECT increment_campaign_balance($1, $2)", campaignID, amount)
            _ = tx.Commit(r.Context())
        }
        w.WriteHeader(200)
    }
}
```

## 5. FRONTEND UI ENGINE & DEPLOYMENT SETUP

GoFundMe UI/UX replica constructed with Next.js 14, Tailwind CSS, and custom donation flow components.

```
// Frontend Checkout Component Overview (CampaignView.tsx) export default function CampaignView({ campaign }: { campaign: CampaignProps }) { const percentage = Math.min(Math.round((campaign.currentAmount / campaign.targetAmount) * 100), 100); return ( <div className="min-h-screen bg-[#fbfbfa] text-gray-900"> <header className="bg-white border-b h-16 flex items-center justify-between px-6"> <span className="text-2xl font-black text-[#02a95c]">fenna</span> <button className="bg-[#02a95c] text-white font-bold px-4 py-2 rounded-full">Start a Fenna</button> </header> <main className="max-w-6xl mx-auto p-6 grid grid-cols-12 gap-8"> <div className="col-span-7 space-y-4"> <h1 className="text-3xl font-extrabold">{campaign.title}</h1> <img src={campaign.coverImageUrl} className="rounded-2xl aspect-video object-cover" /> </div> <div className="col-span-5 bg-white p-6 rounded-2xl border shadow-xl"> <div className="text-3xl font-black">{campaign.currency} {campaign.currentAmount}</div> <div className="w-full bg-gray-100 h-3 rounded-full mt-2"> <div className="bg-[#02a95c] h-3 rounded-full" style={{ width: `${percentage}%` }} /> </div> <button className="w-full bg-[#02a95c] text-white font-extrabold py-4 rounded-2xl mt-4">Donate now</button> </div> </main> </div> ); }
```

## 6. Infrastructure Deployment Blueprint (`render.yaml`)

```
services: - type: web name: fenna-go-backend env: go region: frankfurt buildCommand: go build -o main ./cmd/api startCommand: ./main envVars: - key: PORT value: 8080 - key: DATABASE_URL fromDatabase: name: fenna-db property: connectionString databases: - name: fenna-db databaseName: fenna_logger plan: starter
```